

Realtime Audit Report

ExamForge AI | Generated: 2026-08-02

Executive Summary

The realtime audit identified 10 findings, including 2 Critical, 3 High, 3 Medium, and 2 Low severity issues. The most critical findings are: duplicate realtime subscriptions on the notifications table (both a global provider and a per-page subscription), causing double processing of every event, and no onAuthStateChange listener, meaning the application session desyncs when tokens are refreshed. The offline audit identified 7 findings, including 2 Critical, 2 High, 2 Medium, and 1 Low severity issues. The useOffline hook is never used, Dexie is installed but never configured, and there is no offline mutation queue or network recovery handler.

Category	Critical	High	Medium	Low
Realtime	2	3	3	2
Offline	2	2	2	1

Critical Findings

R-1: Duplicate Realtime Subscriptions on Notifications

Two separate Supabase realtime channels subscribe to the same notifications table with the same user_id filter: (1) RealtimeProvider (global, mounted in app layout) on channel "global-notifications", and (2) NotificationsPage (per-page) on channel "notifications-realtime". When a user navigates to /notifications, both channels are active simultaneously. Every database event triggers two separate handlers, causing double processing, wasted WebSocket bandwidth, and server-side resources. Fix: Remove the per-page subscription and have the page consume from the NotificationStore that the RealtimeProvider already populates.

R-2: No onAuthStateChange Listener

The application never calls supabase.auth.onAuthStateChange(). The useAuthStore is populated during login but never updated when the access token is silently refreshed (typically every hour), when the user signs out from another tab, or when the session expires. The RealtimeProvider reads user from useAuthStore, so if the session changes client-side, the realtime channel will not be re-created with new credentials. This can lead to events silently failing after token expiry. Fix: Add an onAuthStateChange listener in the RealtimeProvider or a dedicated AuthProvider.

O-1: useOffline Hook Never Used

The useOffline hook is well-implemented (detects navigator.onLine, listens for online/offline events, cleans up on unmount) but is never imported or used by any component. There is no offline banner, no connection status indicator, and no disabled state for actions requiring network. Fix: Add an OfflineBanner component in the app layout and use useOffline in the header.

O-2: No Retry Queue - Dexie Not Used

Dexie is listed as a dependency but never configured. There is no IndexedDB database, no offline mutation queue, and no mechanism to sync answers when the network recovers. The

ExamSessionStore persists to localStorage but has no sync logic. Fix: Create a Dexie database for offline storage, wrap mutations with offline-aware logic, and implement a sync engine.

High Severity Findings

R-3: No Reconnection with Data Reconciliation

Both realtime subscriptions log CHANNEL_ERROR and CLOSED status but never attempt to recover missed data. Supabase will reconnect the WebSocket but does not replay missed events. After reconnection, any notifications inserted while disconnected are never delivered. Fix: On reconnection (SUBSCRIBED status after a previous error), re-fetch unread count and notifications list.

R-5: Unbounded Toast Arrays - Memory Leak

Both NotificationStore.toasts and UIStore.toasts arrays grow indefinitely. The addToast action only appends items, and removeToast requires explicit manual invocation. If a burst of notifications arrives (e.g., teacher publishes an exam to 200 students), the array could grow to hundreds of entries. Fix: Cap the arrays at 50 entries and add TTL-based auto-cleanup.

O-3: No Network Recovery Handler

The useOffline hook detects when the network comes back online, but no component acts on this event. There is no handler that replays queued mutations, refetches stale data, or reconciles the cache. Fix: Add a useNetworkRecovery hook that invalidates TanStack Query cache and replays pending mutations from the offline queue.

O-4: No Conflict Resolution Strategy

The ExamSessionStore persists answers to localStorage but has no conflict resolution when a student answers offline and the exam timer expires on the server, or when offline answers conflict with the server version. Fix: Add versioning/timestamps to answers and implement a "last write wins" or "server wins" strategy.

Priority Remediation Order

Priority	Action	Impact
P0	Eliminate duplicate subscriptions	Double processing, wasted bandwidth
P0	Add onAuthStateChange listener	Session desync after token refresh
P0	Wire up useOffline and add offline banner	Users have no offline awareness
P0	Implement Dexie-based offline mutation queue	Offline mutations are silently lost
P1	Cap toast arrays at 50 entries	Memory leak in long-running sessions
P1	Add reconnection data reconciliation	Missed notifications after reconnect
P1	Add network recovery handler	Stale data after network recovery
P1	Add conflict resolution for exam answers	CBT integrity during offline
P2	Fix race condition in notifications setup	Potential duplicate events
P2	Make live indicator reflect connection status	Misleading UX